

# Data Foundation & ML Framework

Understanding Data, Paradigms, and Evaluation

Nipun Batra | IIT Gandhinagar

# Learning Goals

By the end of this lecture, you will be able to:

1. **Distinguish** traditional programming from machine learning
2. **Identify** the three learning paradigms (supervised, unsupervised, RL)
3. **Use** proper mathematical notation for ML
4. **Differentiate** classification from regression
5. **Explain** train/test split and why it matters
6. **Apply** evaluation metrics to measure model performance

# Part 1: Traditional vs ML

The Paradigm Shift

# The Big Question

Every AI/ML system answers **one fundamental question**:

**"Given some input, what should the output be?"**

| Input          | Output           | System           |
|----------------|------------------|------------------|
| Email text     | Spam or Not Spam | Spam Filter      |
| Image          | "Cat" or "Dog"   | Image Classifier |
| House features | Price (₹)        | Price Predictor  |

# Traditional Programming

You write the rules explicitly

```
Input: Email text
Rules: IF "FREE" in email → Spam
      IF "winner" in email → Spam
      IF sender not in contacts AND links > 5 → Spam
      ...
Output: Spam or Not Spam
```

## Problems:

- What about "Fr33" or "w1nner"?
- Rules conflict with each other
- Can't handle every variation

# Machine Learning Approach

Let the computer learn the rules from examples!

```
# Give it examples
emails = [email1, email2, email3, ...]
labels = ["spam", "not_spam", "spam", ...]

# Model learns patterns itself
model.fit(emails, labels)

# Now it can predict new emails
model.predict(new_email) # → "spam"
```

ML excels when patterns are complex or rules are hard to specify explicitly.

# What is Machine Learning?

"A computer program is said to learn from **experience E** with respect to some class of **tasks T** and **performance measure P** if its performance at tasks in T, as measured by P, improves with experience E."

— Tom Mitchell

| Component              | Example: Spam Detection          |
|------------------------|----------------------------------|
| <b>Task (T)</b>        | Classify emails as spam/not spam |
| <b>Experience (E)</b>  | Database of labeled emails       |
| <b>Performance (P)</b> | Accuracy on new emails           |

# Part 2: Learning Paradigms

Three Ways Machines Learn



# Three Learning Paradigms

| Paradigm             | Has Labels? | Learns From            | Example           |
|----------------------|-------------|------------------------|-------------------|
| <b>Supervised</b>    | ✓ Yes       | Input + correct output | Spam detection    |
| <b>Unsupervised</b>  | ✗ No        | Just inputs            | Customer grouping |
| <b>Reinforcement</b> | Rewards     | Actions + feedback     | Game playing      |

# Supervised Learning

Teacher provides correct answers

Training Data:

```
Email_1 → "spam"      (teacher says: spam)
Email_2 → "not_spam"   (teacher says: not spam)
Email_3 → "spam"      (teacher says: spam)
...
```

Model learns: "Hmm, emails with these patterns tend to be spam..."

Test:

```
New_email → ? (model predicts based on learned patterns)
```

This course focuses primarily on supervised learning!

# Unsupervised Learning

No labels — find hidden structure

```
Data: Customer purchase histories (no labels!)
```

```
Algorithm discovers:
```

```
Group A: Buys electronics, tech gadgets
```

```
Group B: Buys groceries, household items
```

```
Group C: Buys books, stationery
```

**Examples:** Clustering, dimensionality reduction, anomaly detection

# Reinforcement Learning

## Learn by trial and error with rewards

Agent (e.g., game AI) takes actions:

Action: Move left → Reward: +1 (good move!)

Action: Move right → Reward: -10 (fell in pit!)

Action: Jump → Reward: +100 (reached goal!)

Agent learns: "Moving right in this situation is bad..."

**Examples:** Game AI (AlphaGo), robotics, autonomous driving

# Focus: Supervised Learning

For most of this course, we'll study supervised learning:

| Task Type             | Output   | Examples                 |
|-----------------------|----------|--------------------------|
| <b>Classification</b> | Category | Spam/Not spam, Cat/Dog   |
| <b>Regression</b>     | Number   | House price, Temperature |

Classification: "Which bucket?" vs Regression: "How much?"

# Part 3: Our First ML Problem

Tomato Quality Prediction

# Problem: Grocery Store Tomato Quality

**Task:** Predict if a tomato is Good or Bad based on visual features

What visual features might be useful?

- Size (Small, Medium, Large)
- Color (Red, Orange, Yellow)
- Texture (Smooth, Rough)

# Our Training Dataset

| Color  | Size   | Texture | Condition |
|--------|--------|---------|-----------|
| Orange | Small  | Smooth  | Good      |
| Red    | Small  | Rough   | Good      |
| Orange | Medium | Smooth  | Bad       |
| Yellow | Large  | Smooth  | Bad       |

**Features (X):** Color, Size, Texture

**Label (y):** Condition (what we want to predict)



# Quick Check: Is Sample Number a Good Feature?

| Sample # | Color  | Size   | Condition |
|----------|--------|--------|-----------|
| 1        | Orange | Small  | Good      |
| 2        | Red    | Small  | Good      |
| 3        | Orange | Medium | Bad       |
| 4        | Yellow | Large  | Bad       |

**Answer:** No! Sample numbers are arbitrary identifiers, not meaningful for prediction.

# Features vs Labels

| Features (X)         | Label (y)       |
|----------------------|-----------------|
| What we observe      | What we predict |
| Inputs               | Output          |
| Color, Size, Texture | Good/Bad        |

```
# In code:  
X = [["Orange", "Small", "Smooth"],  
      ["Red", "Small", "Rough"], ...]  
  
y = ["Good", "Good", "Bad", "Bad"]
```

# Part 4: Mathematical Notation

Speaking the Language of ML

# Why Notation Matters

Computers work with numbers! We need consistent notation.

| Term               | Symbol         | Description                  |
|--------------------|----------------|------------------------------|
| Number of samples  | $n$            | How many examples            |
| Number of features | $d$            | How many input variables     |
| Feature matrix     | $\mathbf{X}$   | All inputs ( $n \times d$ )  |
| Label vector       | $\mathbf{y}$   | All outputs ( $n \times 1$ ) |
| Single sample      | $\mathbf{x}_i$ | i-th input ( $d \times 1$ )  |
| Single label       | $y_i$          | i-th output                  |

# Notation Convention

**Bold UPPERCASE** = Matrix:  $\mathbf{X}$ ,  $\mathbf{W}$

**Bold lowercase** = Vector:  $\mathbf{x}$ ,  $\mathbf{y}$

**Regular** = Scalar:  $n$ ,  $d$ ,  $y_i$

Examples from our tomato dataset:

- $n = 4$  (4 tomatoes)
- $d = 3$  (3 features: Color, Size, Texture)
- $\mathbf{X} \in \mathbb{R}^{4 \times 3}$  (after encoding)
- $\mathbf{y} = [1, 1, 0, 0]^\top$  (Good=1, Bad=0)

# Encoding Categories as Numbers

**Problem:** "Orange", "Red" aren't numbers!

**Solution:** One-hot encoding

| Color  | C_Red | C_Orange | C_Yellow |
|--------|-------|----------|----------|
| Orange | 0     | 1        | 0        |
| Red    | 1     | 0        | 0        |
| Yellow | 0     | 0        | 1        |

```
# In pandas:  
pd.get_dummies(df['Color'])
```

# The Complete Dataset

After encoding, our dataset becomes:

$$\mathbf{X} = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

- $\mathbf{X} \in \mathbb{R}^{4 \times 7}$  (4 samples, 7 features after encoding)
- $\mathbf{y} \in \mathbb{R}^4$  (4 labels)

# The Dataset

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$$

"The dataset  $\mathcal{D}$  is a set of  $n$  pairs, where each pair contains:

- Input features  $\mathbf{x}_i$
- Corresponding label  $y_i$ "

**Example:**  $(\mathbf{x}_1, y_1) = (\text{Orange, Small, Smooth} \rightarrow \text{Good})$



# Part 5: Classification vs Regression

Two Types of Supervised Learning

# Classification: Discrete Outputs

Predict which category

| Type        | Number of Classes | Example          |
|-------------|-------------------|------------------|
| Binary      | 2                 | Spam / Not Spam  |
| Multi-class | $K > 2$           | Cat, Dog, Bird   |
| Multi-label | Multiple          | [Action, Comedy] |

```
model.predict(email)  # → "spam"  
model.predict_proba(email)  # → [0.15, 0.85]  
                           #   [not_spam, spam]
```

# Regression: Continuous Outputs

Predict a number

| Task              | Output Range     | Example       |
|-------------------|------------------|---------------|
| House pricing     | ₹0 - ₹10Cr       | Zillow        |
| Age estimation    | 0 - 100 years    | Face analysis |
| Energy prediction | 0 - $\infty$ kWh | Power grid    |

```
model.predict(house_features) # → 4250000.0 (₹42.5 lakhs)
```

# Quick Classification: Which Type?

| Task                         | Type           | Why?             |
|------------------------------|----------------|------------------|
| "Will it rain tomorrow?"     | Classification | Yes/No           |
| "How many mm of rain?"       | Regression     | Continuous       |
| "What genre is this movie?"  | Classification | Categories       |
| "What rating (1 - 5 stars)?" | Both work!     | Ordered discrete |
| "Which digit (0 - 9)?"       | Classification | 10 categories    |

# Part 6: Training vs Test Split

The Most Important Concept

# The Exam Analogy

Two study strategies:

| Strategy A: Memorize  | Strategy B: Learn   |
|-----------------------|---------------------|
| Q: "2+3=?" A: "5" ✓   | Q: "2+3=?" A: "5" ✓ |
| Q: "2+4=?" A: "???" ✗ | Q: "2+4=?" A: "6" ✓ |

We want ML models to LEARN patterns, not MEMORIZE examples!

# The Train/Test Split

| Training Set     | Test Set               |
|------------------|------------------------|
| Used to learn    | Used to evaluate       |
| Model sees these | Model NEVER sees these |
| 70-80% of data   | 20-30% of data         |

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

# Why Split?

**Without split:** "I got 100% accuracy!" (on training data)

**Reality:** Model memorized the training data

**With split:**

- Training accuracy: 95%
- Test accuracy: 90%
- **This 90% is what matters!** (performance on unseen data)

Test accuracy estimates real-world performance.



# The Golden Rules

## 1. Split BEFORE any processing

```
# ✓ Correct order
X_train, X_test, y_train, y_test = train_test_split(X, y)
scaler.fit(X_train) # Fit ONLY on training
```

## 2. NEVER peek at test data

- Don't use test data for choosing models
- Don't use test data for tuning parameters
- Only use test data for FINAL evaluation

# Part 7: Evaluation Metrics

How Good is Our Model?

# Classification: The Confusion Matrix

Let's say our model predicted on 5 test tomatoes:

| Actual | Predicted | Result    |
|--------|-----------|-----------|
| Good   | Good      | ✓ Correct |
| Good   | Good      | ✓ Correct |
| Bad    | Good      | ✗ Wrong   |
| Bad    | Good      | ✗ Wrong   |
| Bad    | Bad       | ✓ Correct |

**Accuracy = 3/5 = 60%**

# The Four Outcomes

|                  | Predicted: Positive   | Predicted: Negative   |
|------------------|-----------------------|-----------------------|
| Actual: Positive | TP (True Positive) ✓  | FN (False Negative) ✗ |
| Actual: Negative | FP (False Positive) ✗ | TN (True Negative) ✓  |

- **TP:** Correctly predicted positive
- **TN:** Correctly predicted negative
- **FP:** Said positive but was negative (Type I error)
- **FN:** Said negative but was positive (Type II error)

# Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{\text{Correct}}{\text{Total}}$$

**But accuracy can be misleading!**

**Example:** Cancer detection (1% have cancer, 99% healthy)

- Model always predicts "healthy"
- Accuracy = 99%!
- **But it catches 0% of cancer cases!**

# Precision and Recall

**Precision:** Of all predicted positives, how many were actually positive?

$$\text{Precision} = \frac{TP}{TP + FP}$$

**Recall:** Of all actual positives, how many did we catch?

$$\text{Recall} = \frac{TP}{TP + FN}$$

# Precision vs Recall Trade-off

| High Precision                           | High Recall                        |
|------------------------------------------|------------------------------------|
| Predicts positive only when confident    | Catches most positives             |
| Low false alarms ( $\downarrow$ FP)      | Few misses ( $\downarrow$ FN)      |
| May miss some positives ( $\uparrow$ FN) | More false alarms ( $\uparrow$ FP) |

## Which to prefer?

- **Spam filter:** High precision (don't lose important emails!)
- **Cancer test:** High recall (don't miss any cancer cases!)

# F1 Score

Balance between precision and recall:

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- F1 = 1: Perfect (Precision=Recall=1)
- F1 = 0: Terrible (either Precision or Recall is 0)
- Useful when you need both precision and recall



# Regression Metrics

For regression, we measure **how far off** our predictions are:

| Metric      | Formula                                | Interpretation         |
|-------------|----------------------------------------|------------------------|
| <b>MSE</b>  | $\frac{1}{n} \sum (y_i - \hat{y}_i)^2$ | Average squared error  |
| <b>RMSE</b> | $\sqrt{MSE}$                           | Same units as y        |
| <b>MAE</b>  | $\frac{1}{n} \sum  y_i - \hat{y}_i $   | Average absolute error |

**MAE vs MSE:** MAE is less sensitive to outliers

# Regression Example

| Actual (y) | Predicted ( $\hat{y}$ ) | Error |
|------------|-------------------------|-------|
| 100        | 110                     | +10   |
| 200        | 190                     | -10   |
| 150        | 150                     | 0     |

- $MAE = (10 + 10 + 0) / 3 = 6.67$
- $MSE = (100 + 100 + 0) / 3 = 66.67$
- $RMSE = \sqrt{66.67} = 8.16$

# Part 8: The sklearn Pattern

Putting It All Together

# The Universal sklearn Pattern

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# 1. Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# 2. Create model
model = DecisionTreeClassifier()

# 3. Train (fit)
model.fit(X_train, y_train)

# 4. Predict
y_pred = model.predict(X_test)

# 5. Evaluate
print(f"Accuracy: {accuracy_score(y_test, y_pred):.1%}")
```

# The fit/predict Pattern

ALL sklearn models follow the same pattern:

```
# Linear Regression
model = LinearRegression()
model.fit(X_train, y_train)
model.predict(X_test)

# Random Forest
model = RandomForestClassifier()
model.fit(X_train, y_train)
model.predict(X_test)

# Neural Network
model = MLPClassifier()
model.fit(X_train, y_train)
model.predict(X_test)
```

Same 3 methods: `fit()`, `predict()`, `score()`

# Key Takeaways

| Concept           | Key Insight                                           |
|-------------------|-------------------------------------------------------|
| ML vs Programming | ML learns rules from data                             |
| Three Paradigms   | Supervised, Unsupervised, RL                          |
| Notation          | Bold uppercase = matrix, bold lowercase = vector      |
| Classification    | Predict categories                                    |
| Regression        | Predict numbers                                       |
| Train/Test Split  | Never peek at test data!                              |
| Evaluation        | Accuracy isn't everything (use precision, recall, F1) |

# Ready to Build!

Next: Supervised Learning Algorithms

**Lab:** Your first ML models with sklearn

**Interactive Notebook:** [L02\\_data\\_foundation.ipynb](#)

*"In God we trust. All others must bring data."*

— W. Edwards Deming

**Questions?**